

BASICS

Aim: To understand numerical computation and data visualization using NumPy and Matplotlib on the Bank Marketing dataset for exploratory data analysis in Machine Learning applications.

Dataset Source

The dataset used in this experiment is the Bank Marketing Dataset, which contains customer and campaign-related attributes collected from a Portuguese banking institution.

Source:

<https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset>

Dataset Description

The dataset contains customer financial and campaign details used to analyze term deposit subscription behavior.

Dataset Characteristics

Type: Tabular (Numerical + Categorical)

Domain: Banking / Marketing Analytics

Target Variable: **deposit** (Yes/No)

Size: Approximately 11,162 records

Data Quality: Clean dataset with minimal missing values

Important Numerical Features Used

- age
- balance
- duration
- campaign
- pdays
- previous

Mathematical Formulation (NumPy Based Analysis)

Mean

The mean (average) of a numerical feature is calculated as:

$$\text{Mean} = \frac{1}{n} \sum_{i=1}^n x_i$$

Used to calculate:

- Average customer age
- Average account balance

Using NumPy:

```
np.mean()
```

Standard Deviation

Standard deviation measures data dispersion:

$$\sigma = \sqrt{\frac{1}{n} \sum (x_i - \mu)^2}$$

Used to determine:

- Variation in balance
- Variation in campaign duration

Using NumPy:

```
np.std()
```

Minimum and Maximum

$$X_{min} = \min(X), \quad X_{max} = \max(X)$$

Used in normalization and understanding range of financial values.

Using NumPy:

```
np.min(), np.max()
```

Min-Max Normalization

Scaling feature values between 0 and 1:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Used to scale:

- Balance
- Duration
- Age

This helps in preparing data for Machine Learning models.

Algorithm Limitations

- Only descriptive statistical analysis performed.
- No predictive modeling included.
- Visualization patterns may not imply causation.
- Min-max scaling is sensitive to outliers.
- Matplotlib provides basic visualization; complex patterns may require advanced tools.

Methodology / Workflow

1. Load dataset using Pandas
2. Convert numerical columns to NumPy arrays
3. Compute statistical measures:

- Mean
- Standard Deviation
- Min & Max
- 4. Apply Min-Max normalization
- 5. Generate visualizations using Matplotlib:
 - Line plot
 - Histogram
 - Scatter plot
- 6. Interpret trends and data distribution

Performance Analysis

Since no predictive model is trained:

- Balance distribution is highly skewed.
- Age and balance do not show strong linear relationship.
- Histogram shows majority of customers fall under moderate balance category.
- Normalization successfully scales financial data between 0 and 1.

Matplotlib visualizations help in understanding customer financial behavior and data distribution before applying machine learning models.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("Dataset Shape:", df.shape)
print("\nColumn Names:\n", df.columns)
print("\nMissing Values:\n", df.isnull().sum())

print("\nStatistical Summary:\n", df.describe())
```

```
print("\nDeposit Distribution:\n", df['deposit'].value_counts())
```

```
print("\nJob Wise Average Balance:\n")  
print(df.groupby('job')['balance'].mean())
```

```
age_array = df['age'].to_numpy()
```

```
print("\nNumPy Analysis on Age")  
print("Mean Age:", np.mean(age_array))  
print("Max Age:", np.max(age_array))  
print("Min Age:", np.min(age_array))  
print("Variance:", np.var(age_array))
```

```
df['deposit_encoded'] = df['deposit'].map({'yes':1, 'no':0})
```

```
df['Balance_Level'] = pd.cut(df['balance'],  
                             bins=[-np.inf, 0, 2000, 5000, np.inf],  
                             labels=['Negative', 'Low', 'Medium', 'High'])
```

```
print("\nNew Columns Added:")  
print(df[['balance', 'Balance_Level']].head())
```

```
# Pie Chart
```

```
plt.figure()  
df['deposit'].value_counts().plot.pie(autopct='%1.1f%%')  
plt.title("Deposit Subscription Percentage")  
plt.ylabel("")  
plt.show()
```

```
# Bar Chart - Job vs Deposit
```

```
plt.figure(figsize=(10,5))  
df.groupby('job')['deposit_encoded'].mean().plot(kind='bar')  
plt.title("Job vs Subscription Rate")  
plt.ylabel("Average Subscription Rate")  
plt.xticks(rotation=45)  
plt.show()
```

```
# KDE Plot
```

```
plt.figure()
```

```
sns.kdeplot(data=df, x='age', hue='deposit', fill=True)
plt.title("Age Distribution by Deposit")
plt.show()
```

```
# Pairplot (Only Important Columns)
sns.pairplot(df[['age', 'balance', 'duration', 'deposit']],
             hue='deposit')
plt.show()
```

```
# Correlation Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="viridis")
plt.title("Correlation Matrix")
plt.show()
```

```
bank.csv(text/csv) - 918960 bytes, last modified: 2/11/2026 - 100% done
Saving bank.csv to bank (5).csv
Dataset Shape: (11162, 17)

Column Names:
Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',
       'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'deposit'],
      dtype='object')

Missing Values:
age      0
job      0
marital  0
education 0
default  0
balance  0
housing  0
loan     0
contact  0
day      0
month    0
duration 0
campaign 0
pdays   0
previous 0
poutcome 0
deposit  0
dtype: int64

Statistical Summary:

```

	age	balance	day	duration	campaign \
count	11162.000000	11162.000000	11162.000000	11162.000000	11162.000000
mean	41.231948	1528.538524	15.658036	371.993818	2.508421
std	11.913369	3225.413326	8.420740	347.128386	2.722077
min	18.000000	-6847.000000	1.000000	2.000000	1.000000
25%	32.000000	122.000000	8.000000	138.000000	1.000000
50%	39.000000	550.000000	15.000000	255.000000	2.000000
75%	49.000000	1708.000000	22.000000	496.000000	3.000000
max	95.000000	81204.000000	31.000000	3881.000000	63.000000

	pdays	previous
count	11162.000000	11162.000000
mean	51.330407	0.832557
std	108.758282	2.292007
min	-1.000000	0.000000
25%	-1.000000	0.000000
50%	-1.000000	0.000000
75%	20.750000	1.000000
max	854.000000	58.000000

Deposit Distribution:

deposit	count
no	5873
yes	5289

Name: count, dtype: int64

Job Wise Average Balance:

job	balance
admin.	1195.866567
blue-collar	1203.926440
entrepreneur	1621.942073
housemaid	1366.160584
management	1793.663679
retired	2417.250643
self-employed	1865.372840
services	1081.171181
student	1500.783333
technician	1556.294569
unemployed	1314.719888
unknown	1945.457143

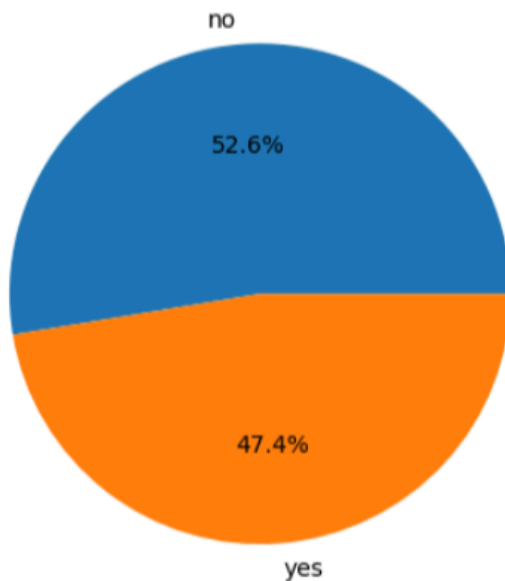
Name: balance, dtype: float64

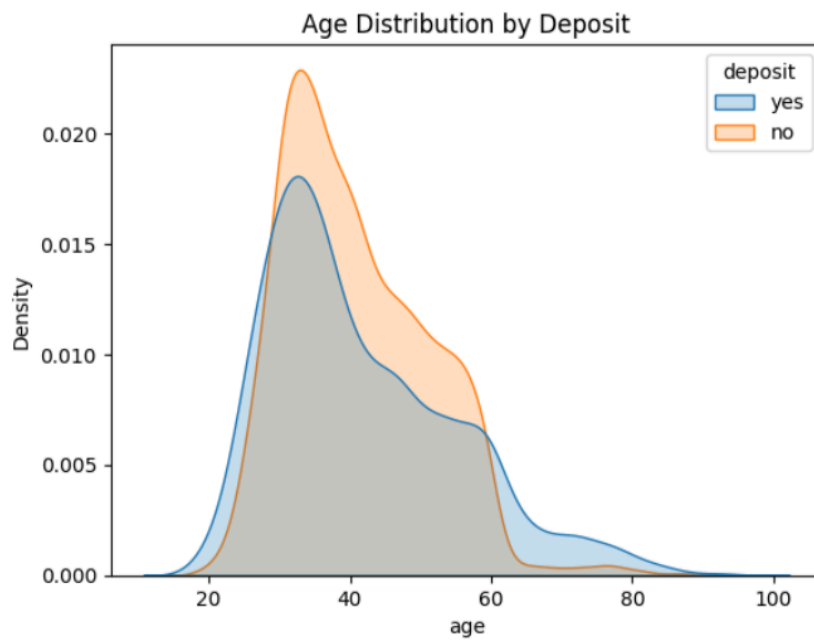
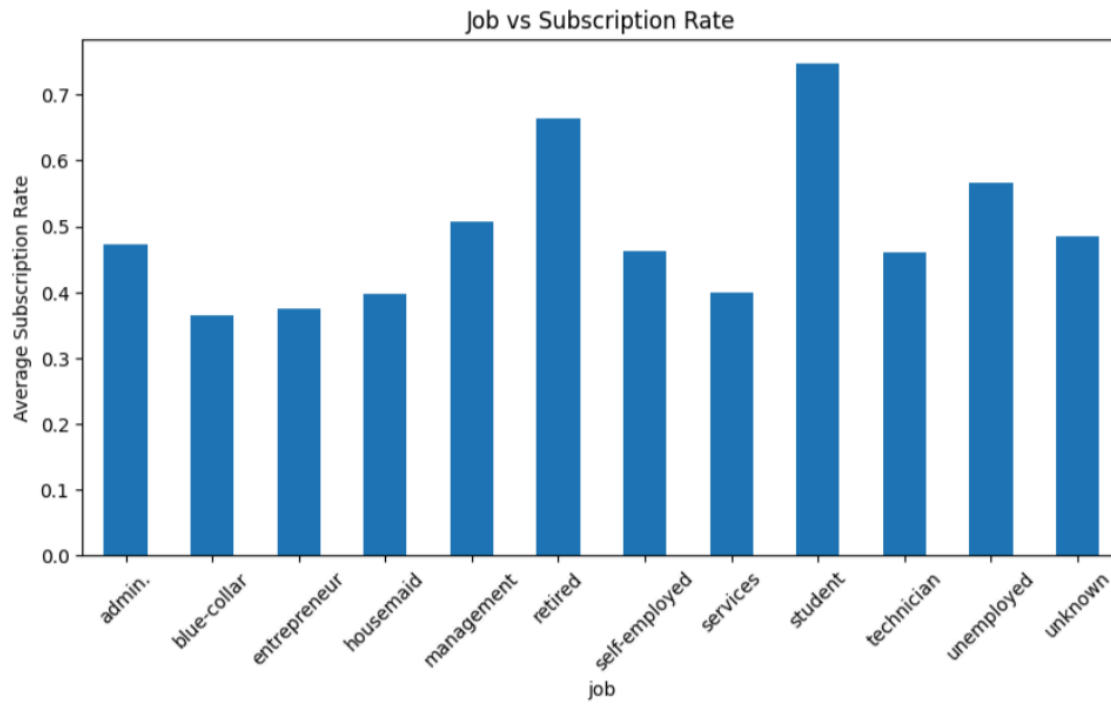
NumPy Analysis on Age
Mean Age: 41.231947679627304
Max Age: 95
Min Age: 18
Variance: 141.9156501932848

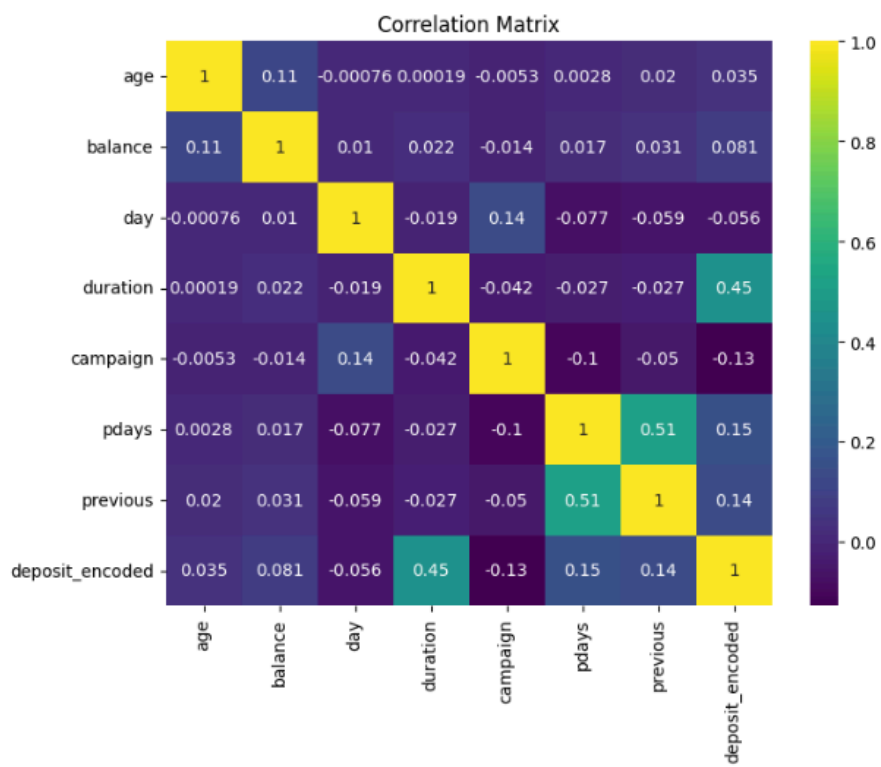
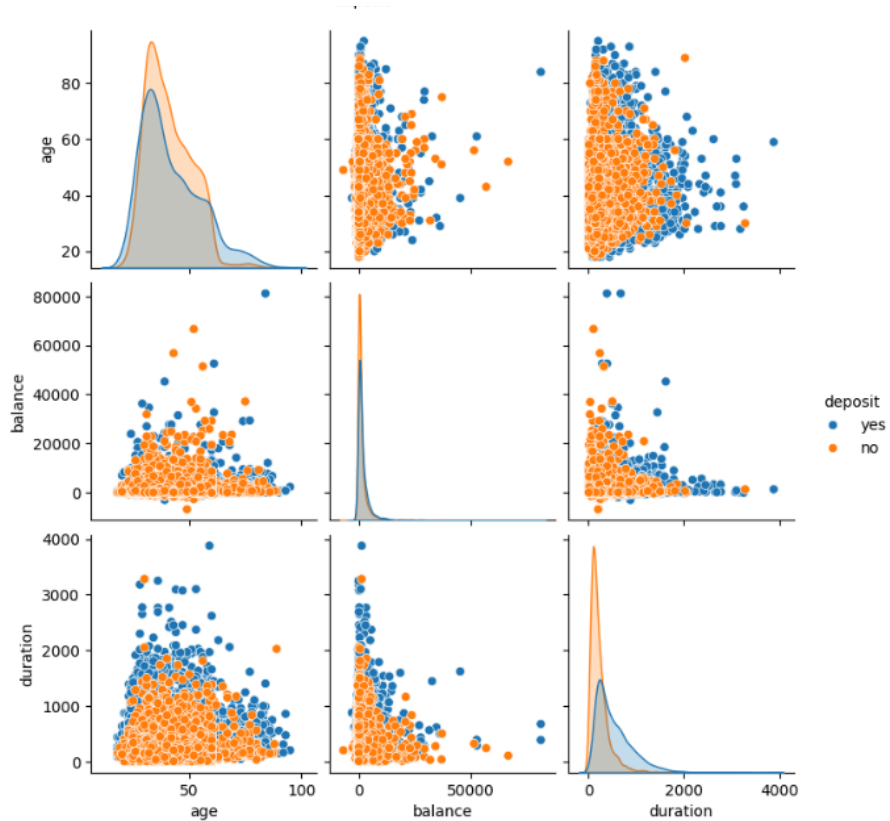
New Columns Added:

	balance	Balance_Level
0	2343	Medium
1	45	Low
2	1270	Low
3	2476	Medium
4	184	Low

Deposit Subscription Percentage







Conclusion

This experiment demonstrated the use of:

- **NumPy** for numerical computation and statistical analysis.
- **Matplotlib** for graphical visualization of data trends.

Key Insights:

- Statistical measures help summarize customer financial behavior.
- Visualization helps understand data distribution and relationships.
- Preprocessing like normalization is essential before applying machine learning models.

Thus, NumPy and Matplotlib form the foundation for data analysis in Machine Learning workflows.